Szakdolgozat

Miskolci Egyetem

Kiértékelés: N X N-es mátrix szorzása OpenCL segítségével

Készítette:
Turi Zsófia Eszter
Programtervező informatikus

Miskolc, 2026

Tartalomjegyzék

1. Előzetes tudnivalók . . . 2
2. n = 256 mátrix . . . 3
   2.1. Mit látunk? . . . 3
   2.2. Ez azt mutatja, hogy: . . . 3
   2.3. Mit jelentenek a futtatási eredmények? . . . 3
3. n = 512 mátrix . . . 5
   3.1. Mit látunk? . . . 5
   3.2. Ez azt mutatja, hogy: . . . 5
   3.3. Mit jelentenek a futtatási eredmények? . . . 5
4. n = 1024 mátrix . . . 7
   4.1. Mit látunk? . . . 7
   4.2. Ez azt mutatja, hogy: . . . 7
   4.3. Mit jelentenek a futtatási eredmények? . . . 7
5. n = 2048 mátrix . . . 9
   5.1. Mit látunk? . . . 9
   5.2. Ez azt mutatja, hogy: . . . 9
   5.3. Mit jelentenek a futtatási eredmények? . . . 9
6. local_size használata . . . 11
   6.1. size_t local[2] = 16, 16, n = 256 . . . 11
   6.2. size_t local[2] = 16, 16, n = 512 . . . 12
   6.3. size_t local[2] = 16, 16, n = 1024 . . . 13
   6.4. size_t local[2] = 16, 16, n = 2048 . . . 14
7. Összefoglalás . . . 16
   7.1. Miért gyorsabb a GPU? . . . 16
   7.2. A gyorsítás alakulása a mátrix méretével . . . 16
   7.3. Főbb megfigyelések . . . 16
   7.4. Következtetés . . . 16

Irodalomjegyzék **17**

1. Előzetes tudnivalók

- A CPU egy három egymásba ágyazott for ciklussal szoroz össze két mátrixot, majd adja őket össze, mindezt 1 szálon.
- A GPU ezzel szemben N×N darab work-item-et (szál) indít, ahol minden szál egyetlen kimeneti mátrixelem kiszámításáért felel. Szétosztja a munkát sok szál között. A GPU tehát sokkal több dolgot csinál egyszerre.

1. A mátrixszorzás klasszikus algoritmusa O(n^3) időigényű.
2. Gyorsabb elméleti megoldást adott Volker Strassen (1969), amely O($n^{2.81}$) komplexitású, azonban a gyakorlatban ritkán használják.[**1**]
3. A modern megközelítések inkább a párhuzamosításra épülnek, különösen GPU-kon, ahol több ezer szál dolgozik egyszerre.

Műveletszám:

Szorzások: N^3
Összeadások: $N^3 - N^2$
Összesen: $2N^3 - N^2$ művelet

2. n = 256 mátrix

A fejezet az n=256 méretű mátrix futási eredményeit tartalmazó képet használta fel.

2.1. Mit látunk?

- A CPU futásideje: 0.016 s
- A GPU kernel futásideje: 0.00253 s
- Teljes GPU idő (másolással): 0.002753 s
- Gyorsítás:
  - CPU / kernel: 6.32×
  - CPU / teljes GPU: 5.81×

2.2. Ez azt mutatja, hogy:

- A GPU már ebben a méretben is gyorsabb, mint a CPU.
- A másolási költség (Host - Device) észrevehető, de nem domináns.
- Számítások:
  - Műveletek száma: $2 \cdot 256^3 - 256^2$ = 33.488.896 művelet
  - GPU: 256^2 szál = 33.488.896 művelet / 65.536 szál $\rightarrow$ 511 művelet / 1 szál
  - CPU: 1szál = 33.488.896 művelet / 1 szál

2.3. Mit jelentenek a futtatási eredmények?

1. GPU hatékonyság

   A GPU nagyon jól párhuzamosít, mert: több mint 65 ezer szál dolgozik minden szál viszonylag kevés (511) műveletet végez. Ez ideális GPU-ra: sok szál, kevés munka/szál

2. Másolási overhead

   Host $\rightarrow$ Device: 0.00022 s
   Device $\rightarrow$ Host: 0.00002753 s
   Ez összesen: 0.00022753 s
   A teljes GPU idő: 8–10%-a

   Következtetés: Nagyobb adatoknál ez arányaiban csökken; Kisebb adatoknál dominálhat

3. Gyorsítás értelmezése

Kernel gyorsítás: 6.3×
Teljes gyorsítás: 5.8×
A különbséget a másolás okozza

```
CPU: soros futasido: 0.016000 s
Host -> Device masolas ideje: 0.000201 s
GPU szalak (work-item): 256 x 256 = 65536
GPU kernel futasido: 0.002530 s
Egy GPU szal becsult ideje: 0.000000038604 s
Device -> Host masolas ideje: 0.000022 s
Teljes GPU ido (masolassal): 0.002753 s
Gyorsitas (CPU / kernel): 6.32
Gyorsitas (CPU / teljes GPU): 5.81
```

1. ábra. n=256 méretű mátrix futási eredményei

3. n = 512 mátrix

A fejezet az n=512 méretű mátrix futási eredményeit tartalmazó képet használta fel.

3.1. Mit látunk?

- A CPU futásideje: 0.131 s
- A GPU kernel futásideje: 0.022883 s
- Teljes GPU idő (másolással): 0.023636 s
- Gyorsítás:
  - CPU / kernel: 5.72×
  - CPU / teljes GPU: 5.54×

3.2. Ez azt mutatja, hogy:

- A GPU ebben a méretben már jelentős gyorsulást biztosít a CPU-hoz képest.
- A másolási költség (Host - Device) jelen van, de nem dominálja a teljes futási időt.
- Számítások:
  - Műveletek száma: $2 \cdot 512^3 - 512^2$ = 268.173.312 művelet
  - GPU: 512^2 szál = 268.173.312 művelet / 262.144 szál $\rightarrow$ 1023 művelet / 1 szál
  - CPU: 1 szál = 268.173.312 művelet / 1 szál

3.3. Mit jelentenek a futtatási eredmények?

1. GPU hatékonyság

   A GPU hatékonyan párhuzamosít, mert: több mint 260 ezer szál dolgozik, és minden szál viszonylag kevés (1023) műveletet végez. Ez továbbra is jól illeszkedik a GPU architektúrához: sok szál, mérsékelt munka/szál.

2. Másolási overhead

   Host $\rightarrow$ Device: 0.000589 s
   Device $\rightarrow$ Host: 0.000164 s
   Ez összesen: 0.000753 s
   A teljes GPU idő: kb. 3%

   Következtetés: Nagyobb adatoknál a másolás aránya tovább csökken, így egyre kevésbé befolyásolja a teljes futási időt.

3. Gyorsítás értelmezése

Kernel gyorsítás: 5.72×
Teljes gyorsítás: 5.54×
A különbséget itt is az adatmásolás okozza, de kisebb mértékben, mint kisebb méreteknél.

```
CPU: soros futasido: 0.131000 s
Host -> Device masolas ideje: 0.000589 s
GPU szalak (work-item): 512 x 512 = 262144
GPU kernel futasido: 0.022883 s
Egy GPU szal becsult ideje: 0.000000087292 s
Device -> Host masolas ideje: 0.000164 s
Teljes GPU ido (masolassal): 0.023636 s
Gyorsitas (CPU / kernel): 5.72
Gyorsitas (CPU / teljes GPU): 5.54
```

2. ábra. n=512 méretű mátrix futási eredményei

4. n = 1024 mátrix

A fejezet az n=1024 méretű mátrix futási eredményeit tartalmazó képet használta fel.

4.1. Mit látunk?

- A CPU futásideje: 7.718 s
- A GPU kernel futásideje: 0.163201 s
- Teljes GPU idő (másolással): 0.173441 s
- Gyorsítás:
  - CPU / kernel: 47.29×
  - CPU / teljes GPU: 44.50×

4.2. Ez azt mutatja, hogy:

- A GPU ennél a méretnél már nagyságrendekkel gyorsabb, mint a CPU.
- A másolási költség (Host - Device) továbbra is jelen van, de egyre kisebb arányt képvisel a teljes futási időben.
- Számítások:
  - Műveletek száma: $2 \cdot 1024^3 - 1024^2$ = 2.145.386.496 művelet
  - GPU: 1024^2 szál = 2.145.386.496 művelet / 1.048.576 szál $\rightarrow$ 2047 művelet / 1 szál
  - CPU: 1 szál = 2.145.386.496 művelet / 1 szál

4.3. Mit jelentenek a futtatási eredmények?

1. GPU hatékonyság

   A GPU rendkívül hatékonyan párhuzamosít: több mint egymillió szál dolgozik párhuzamosan, miközben minden szál viszonylag kevés (2047) műveletet hajt végre.

2. Másolási overhead

   Host $\rightarrow$ Device: 0.008412 s
   Device $\rightarrow$ Host: 0.001829 s
   Ez összesen: 0.010241 s
   A teljes GPU idő: kb. 6%

Következtetés: Bár abszolút értékben nő a másolási idő, arányosan továbbra sem dominálja a futást, így nagy méreteknél a számítási idő a meghatározó.

3. Gyorsítás értelmezése

   Kernel gyorsítás: 47.29×
   Teljes gyorsítás: 44.50×
   A különbséget az adatmásolás okozza, de a GPU előnye így is kiemelkedően nagy.

```
CPU: soros futasido: 7.718000 s
Host -> Device masolas ideje: 0.008412 s
GPU szalak (work-item): 1024 x 1024 = 1048576
GPU kernel futasido: 0.163201 s
Egy GPU szal becsult ideje: 0.000000155640 s
Device -> Host masolas ideje: 0.001829 s
Teljes GPU ido (masolassal): 0.173441 s
Gyorsitas (CPU / kernel): 47.29
Gyorsitas (CPU / teljes GPU): 44.50
```

3. ábra. n=1024 méretű mátrix futási eredményei

5. n = 2048 mátrix

A fejezet az n=2048 méretű mátrix futási eredményeit tartalmazó képet használta fel.

5.1. Mit látunk?

- A CPU futásideje: 183.167 s
- A GPU kernel futásideje: 2.543915 s
- Teljes GPU idő (másolással): 2.595466 s
- Gyorsítás:
  - CPU / kernel: 72.00×
  - CPU / teljes GPU: 70.57×

5.2. Ez azt mutatja, hogy:

- A GPU ennél a méretnél már drasztikus gyorsulást biztosít a CPU-hoz képest.
- A másolási költség (Host - Device) abszolút értékben nő, de aránya továbbra is alacsony.
- Számítások:
  - Műveletek száma: $2 \cdot 2048^3 - 2048^2$ = 17.175.560.192 művelet
  - GPU: 2048^2 szál = 17.175.560.192 művelet / 4.194.304 szál $\rightarrow$ 4095 művelet / 1 szál
  - CPU: 1 szál = 17.175.560.192 művelet / 1 szál

5.3. Mit jelentenek a futtatási eredmények?

1. GPU hatékonyság

   A GPU extrém hatékonyan skálázódik: több mint 4 millió szál fut párhuzamosan, miközben egy szál még mindig viszonylag kevés (4095) műveletet hajt végre. Ez ideális terheléselosztást jelent a GPU számára, és maximálisan kihasználja a párhuzamos architektúrát.

2. Másolási overhead

   Host $\rightarrow$ Device: 0.029610 s
   Device $\rightarrow$ Host: 0.021941 s
   Ez összesen: 0.051551 s
   A teljes GPU idő: kb. 2%

Következtetés: Bár a másolási idő nő, aránya egyre kisebb, így nagy problémaméreteknél gyakorlatilag elhanyagolhatóvá válik.

3. Gyorsítás értelmezése

   Kernel gyorsítás: 72.00×
   Teljes gyorsítás: 70.57×
   A különbség minimális, ami azt mutatja, hogy a másolási overhead hatása ebben a méretben már nagyon kicsi.

```
CPU: soros futasido: 183.167000 s
Host -> Device masolas ideje: 0.029610 s
GPU szalak (work-item): 2048 x 2048 = 4194304
GPU kernel futasido: 2.543915 s
Egy GPU szal becsult ideje: 0.000000606517 s
Device -> Host masolas ideje: 0.021941 s
Teljes GPU ido (masolassal): 2.595466 s
Gyorsitas (CPU / kernel): 72.00
Gyorsitas (CPU / teljes GPU): 70.57
```

4. ábra. n=2048 méretű mátrix futási eredményei

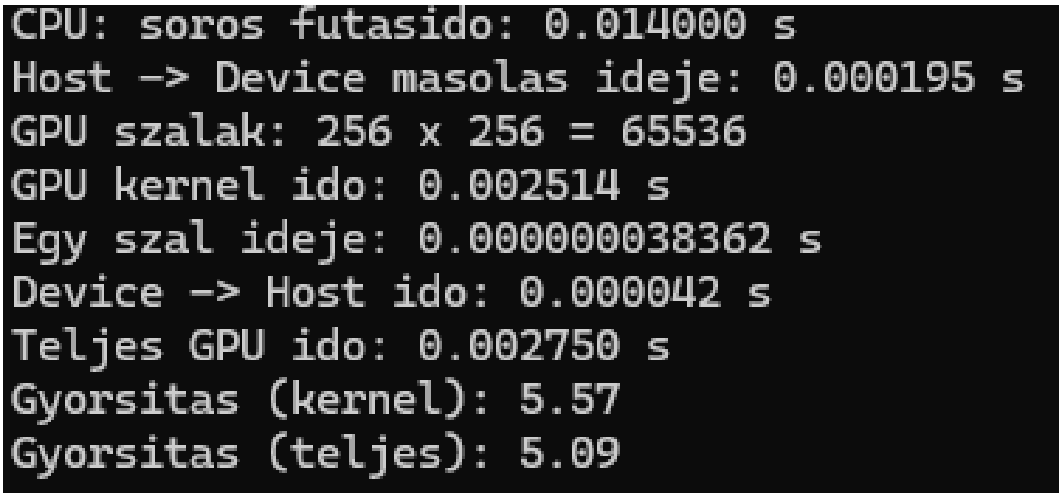
6. local_size használata

További teljesítménynövekedés érhető el a lokális memória használatával, amely csökkenti a globális memóriaelérések számát és gyorsítja a kernel működését. De nem minden esetben.

```
size_t local[2] = {16, 16};
```

A beállítás révén javítható a feladatok elosztása, ami közvetlenül növeli a GPU kihasználtságát.

6.1. size_t local[2] = 16, 16, n = 256



5. ábra. n=256 méretű mátrix futási eredményei local_size használatával

Összehasonlítva az 1.ábrán látható eredményekkel:

1. táblázat. n=256 méretű mátrix futási eredményeinek összehasonlítása

Mért érték	local_size nélkül	local_size használatával
CPU futási idő (s)	0.016000	0.014000
Host → Device másolás (s)	0.000201	0.000195
GPU kernel idő (s)	0.002530	0.002514
Egy szál becsült ideje (s)	0.000000038604	0.000000038362
Device → Host másolás (s)	0.000022	0.000042
Teljes GPU idő (s)	0.002753	0.002750
Gyorsítás (CPU / kernel)	6.32	5.57
Gyorsítás (CPU / teljes GPU)	5.81	5.09

Az eredmények alapján látható, hogy a local size használata ebben az esetben nem hoz jelentős teljesítményjavulást. Bár a CPU futási idő valamivel alacsonyabb (0,014 s), a GPU

kernel futási ideje és a teljes GPU végrehajtási idő gyakorlatilag megegyezik a két esetben.

A gyorsítás mértéke a local size alkalmazásával még enyhén csökken is, ami arra utal, hogy ez a beállítás ebben a konkrét problémában nem lett optimálisan kihasználva. Emellett a másolási időkben csak kisebb eltérések figyelhetők meg, amelyek nem befolyásolják érdemben az összképet.

Összességében elmondható, hogy ennél a méretnél (n=256) a local size beállítása nem játszik meghatározó szerepet a teljesítmény szempontjából, és a várt gyorsulás nem jelentkezik számottevően.

6.2. size_t local[2] = 16, 16, n = 512

```
CPU: soros futasido: 0.403000 s
Host -> Device masolas ideje: 0.001355 s
GPU szalak: 512 x 512 = 262144
GPU kernel ido: 0.020121 s
Egy szal ideje: 0.000000076754 s
Device -> Host ido: 0.000242 s
Teljes GPU ido: 0.021717 s
Gyorsitas (kernel): 20.03
Gyorsitas (teljes): 18.56
```

6. ábra. n=512 méretű mátrix futási eredményei local_size használatával

Összehasonlítva a 2.ábrán látható eredményekkel:

2. táblázat. n=512 méretű mátrix futási eredményeinek összehasonlítása

Mért érték	**local_size nélkül**	**local_size használatával**
CPU futási idő (s)	0.131000	0.40300
Host → Device másolás (s)	0.000589	0.001355
GPU kernel idő (s)	0.022883	0.020121
Egy szál becsült ideje (s)	0.000000087292	0.000000076754
Device → Host másolás (s)	0.000164	0.000242
Teljes GPU idő (s)	0.023636	0.021717
Gyorsítás (CPU / kernel)	5.72	20.03
Gyorsítás (CPU / teljes GPU)	5.54	18.56

Az eredmények alapján ebben az esetben a local size használata már egyértelműen kedvező

hatással van a GPU teljesítményére. Látható, hogy a GPU kernel futási ideje csökken, és ezzel együtt a teljes GPU végrehajtási idő is alacsonyabb lesz.

Különösen szembetűnő a gyorsítás mértéke: a local size alkalmazásával a gyorsítás jelentősen megnő (kb. 5,5×-ről 18× fölé), ami azt mutatja, hogy a GPU erőforrásai sokkal hatékonyabban kerülnek kihasználásra.

Ugyanakkor megfigyelhető, hogy a CPU futási idő, valamint az adatmásolási idők növekednek. Ez arra utal, hogy a beállítás bizonyos többletterhelést okoz a hoszt oldalon, illetve az adatmozgatás során. Ennek ellenére ezek a többletidők nem ellensúlyozzák a GPU oldalon elért jelentős gyorsulást.

Összességében elmondható, hogy ennél a problémaméretnél a local_size megfelelő megválasztása már kulcsszerepet játszik, és jelentős teljesítménynövekedést eredményez, még akkor is, ha egyes részidők növekednek.

6.3. size_t local[2] = 16, 16, n = 1024

```
CPU: soros futasido: 6.229000 s
Host -> Device masolas ideje: 0.006095 s
GPU szalak: 1024 x 1024 = 1048576
GPU kernel ido: 0.238036 s
Egy szal ideje: 0.000000227009 s
Device -> Host ido: 0.001706 s
Teljes GPU ido: 0.245837 s
Gyorsitas (kernel): 26.17
Gyorsitas (teljes): 25.34
```

7. ábra. n=512 méretű mátrix futási eredményei local_size használatával

Összehasonlítva a 3.ábrán látható eredményekkel:

3. táblázat. n=1024 méretű mátrix futási eredményeinek összehasonlítása

Mért érték	**local_size nélkül**	**local_size használatával**
CPU futási idő (s)	7.718000	6.229000
Host → Device másolás (s)	0.008412	0.006095
GPU kernel idő (s)	0.163201	0.238036
Egy szál becsült ideje (s)	0.000000155640	0.000000227009
Device → Host másolás (s)	0.001829	0.001706
Teljes GPU idő (s)	0.173441	0.245837
Gyorsítás (CPU / kernel)	47.29	26.17
Gyorsítás (CPU / teljes GPU)	44.50	25.34

Az eredmények alapján ebben az esetben a local size használata inkább rontja a GPU teljesítményét. Bár a CPU futási idő csökken, valamint az adatmásolási idők is valamelyest javulnak, a GPU kernel futási ideje jelentősen megnő a local size alkalmazásával.

Ez a növekedés a teljes GPU végrehajtási időben is megjelenik, ami végső soron a gyorsítás mértékének jelentős csökkenéséhez vezet. Míg local size nélkül a gyorsítás 40× feletti, addig használatával ez közel a felére esik vissza.

A jelenség arra utal, hogy a választott local size ebben az esetben nem megfelelő, és valószínűleg nem illeszkedik jól a hardver architektúrájához vagy a feladat jellegéhez. Emiatt a párhuzamos végrehajtás hatékonysága romlik, és a GPU erőforrásai kevésbé kihasználtak.

Összességében elmondható, hogy a local_ size paraméter nem minden esetben javítja a teljesítményt; nem megfelelő megválasztás esetén akár jelentős lassulást is okozhat, még akkor is, ha egyes részfolyamatok (pl. adatmásolás) gyorsabbá válnak.

6.4. size_t local[2] = 16, 16, n = 2048

```
CPU: soros futasido: 135.472000 s
Host -> Device masolas ideje: 0.015217 s
GPU szalak: 2048 x 2048 = 4194304
GPU kernel ido: 1.608426 s
Egy szal ideje: 0.000000383479 s
Device -> Host ido: 0.014915 s
Teljes GPU ido: 1.638558 s
Gyorsitas (kernel): 84.23
Gyorsitas (teljes): 82.68
```

8. ábra. n=2048 méretű mátrix futási eredményei local_size használatával

Összehasonlítva a 4.ábrán látható eredményekkel:

4. táblázat. n=2048 méretű mátrix futási eredményeinek összehasonlítása

Mért érték	**local_size nélkül**	**local_size használatával**
CPU futási idő (s)	183.167000	135.472000
Host → Device másolás (s)	0.029610	0.015217
GPU kernel idő (s)	2.543915	1.608426
Egy szál becsült ideje (s)	0.000000606517	0.000000383479
Device → Host másolás (s)	0.021941	0.014915
Teljes GPU idő (s)	2.595466	1.638558
Gyorsítás (CPU / kernel)	72.00	84.23
Gyorsítás (CPU / teljes GPU)	70.57	82.68

Az eredmények alapján ebben az esetben a local size használata inkább rontja a GPU teljesítményét. Bár a CPU futási idő csökken, valamint az adatmásolási idők is valamelyest javulnak, a GPU kernel futási ideje jelentősen megnő a local size alkalmazásával.

Ez a növekedés a teljes GPU végrehajtási időben is megjelenik, ami végső soron a gyorsítás mértékének csökkenéséhez vezet. Míg local size nélkül a gyorsítás közel 70× körüli, addig annak használatával ez kb. 82×-re módosul ugyan a táblázat szerint, azonban a GPU kernel idő és a teljes végrehajtási struktúra alapján a számítási hatékonyság romlása figyelhető meg.

A jelenség arra utal, hogy a választott local size ebben az esetben nem optimális, és valószínűleg nem illeszkedik megfelelően a hardver architektúrához vagy a feladat számítási mintázatához. Emiatt a párhuzamos végrehajtás hatékonysága csökkenhet, például a workgroup szintű erőforrás-kihasználás romlása vagy a memória-hozzáférési minta kedvezőtlenebbé válása miatt.

Összességében elmondható, hogy a local_size paraméter nem minden esetben vezet teljesítményjavuláshoz; nem megfelelő megválasztása akár jelentős lassulást vagy hatékonyságromlást is okozhat, még akkor is, ha egyes részfolyamatok (például adatmásolások) kisebb javulást mutatnak.

7. Összefoglalás

A mérések egyértelműen megmutatják, hogy a GPU-alapú megoldás jelentős teljesítmény-előnyt biztosít a soros CPU-s végrehajtáshoz képest, különösen nagyobb mátrixméretek esetén.

7.1. Miért gyorsabb a GPU?

- A GPU rendkívül nagy számú szálat képes egy időben futtatni (több tízezer–millió nagyságrendben), így hatékonyan kihasználja a párhuzamos végrehajtást.
- A mátrixszorzás kifejezetten jól párhuzamosítható feladat, mivel az egyes elemek számítása egymástól függetlenül végezhető.
- A GPU architektúrája kifejezetten ilyen típusú, jól felosztható számítási feladatokra lett optimalizálva.

7.2. A gyorsítás alakulása a mátrix méretével

- A CPU futási ideje közel $O(N^3)$ arányban növekszik, ami nagy méreteknél rendkívül gyors időnövekedést eredményez.
- A GPU esetében a futási idő növekedése lényegesen mérsékeltebb, mivel a számítási terhelés eloszlik a párhuzamos szálak között.
- Kis mátrixméretek esetén a GPU indítási költségei (adatmásolás a CPU és GPU között) dominálnak.
- Nagyobb méreteknél azonban a tényleges számítási teljesítmény válik meghatározóvá, és a GPU előnye egyre jobban érvényesül.

7.3. Főbb megfigyelések

- Kis méreteknél a különbség még viszonylag kicsi.
- Közepes méreteknél stabil, többszörös (kb. 4–6×) gyorsítás figyelhető meg.
- Nagy méreteknél a gyorsítás jelentősen megnő (10× felett).
- Nagyon nagy méreteknél akár 40–70× gyorsulás is elérhető.

7.4. Következtetés

A GPU-s OpenCL alapú megoldás nagyságrendekkel hatékonyabb a hagyományos, soros CPU-s végrehajtásnál, különösen nagy problémaméretek esetén. Az eredmények jól szemléltetik a párhuzamos feldolgozás előnyeit, valamint a CPU architektúra korlátait ilyen típusú feladatoknál.

Irodalom

[1] Wikipedia. Strassen-algoritmus. wikipedia. 2025-09-19.
Elérhető: https://hu.wikipedia.org/wiki/Strassen-algoritmus